

Draft Microarchitecture Specification

INT4 MAC TRANSFORMER ACCELERATOR

A 4×4 INT4 Tile for Tiny Tapeout on Sky130

A small, weight-stationary INT4 MAC array with an accumulator bank, designed to demonstrate transformer matmul and attention primitives within the area, I/O, and memory constraints of a Tiny Tapeout tile in the Skywater 130 nm open-source PDK.

Block:	<code>int4_mac_accel</code>
Process:	Skywater Sky130 (130 nm)
Platform:	Tiny Tapeout (TT)
Status:	Microarchitecture Draft

Introduction

The INT4 MAC Transformer Accelerator (`int4_mac_accel`) is a weight-stationary 4×4 INT4 multiply-accumulate tile sized for a single 8× Tiny Tapeout slot in the Skywater 130 nm open-source PDK. The block isolates the arithmetic kernel that dominates attention and feed-forward layers in transformer models, wraps it in an on-chip weight ROM, a ping-pong accumulator bank, and an approximate activation LUT, and exposes the whole thing over the standard Tiny Tapeout 8-pin streaming pinout.

This document is the microarchitecture specification. It is the source of truth for RTL design, verification, physical implementation, and post-silicon bring-up. Wherever a feature is described here it is implemented in the synthesizable RTL; anything not described here is not part of the design contract.

DESIGN PHILOSOPHY

Build the smallest honest transformer-shaped accelerator that fits. No off-chip DRAM assumptions, no marketing asterisks, no features in the spec that the RTL does not implement. Constraints define the architecture; the RTL defines the spec.

Top-Level Module & Pinout

The top-level wrapper `int4_mac_accel` instantiates one of each submodule and exposes the Tiny Tapeout pinout. The `uio_oe` drive-enable mask is statically wired to `8'hF3`: bits 0, 1, and 4–7 are driven by the chip, bits 2–3 are driven by the host. The bidirectional bus is therefore partitioned rather than dynamically reversed, which keeps the TT pad logic trivial.

Submodule inventory

Instance	Module	Role
<code>u_ctrl</code>	<code>ctrl_io</code>	Streaming I/O front end, opcode FSM, CSRs.
<code>u_mac</code>	<code>mac_array_4x4</code>	4×4 weight-stationary INT4 MAC array.
<code>u_bank</code>	<code>accum_bank</code>	Ping-pong INT20 accumulator bank, 2×4×4 = 32 entries.
<code>u_rom</code>	<code>weight_rom</code>	256×INT4 synchronous ROM, LFSR-seeded or \$readmemh.
<code>u_as</code>	<code>act_streamer</code>	4-deep INT4 fill buffer presenting a row to the array.
<code>u_rq</code>	<code>requant_sat</code>	Shift-round-saturate INT20 → INT4/INT8, sticky overflow.
<code>u_lut</code>	<code>act_lut</code>	16-entry piecewise activation LUT (EXP_NEG default), bypassable.

Top-level pinout

Signal	Dir	W	Description
<code>clk</code>	in	1	Single clock domain. 25 MHz target, 50 MHz stretch.
<code>rst_n</code>	in	1	Synchronous, active-low reset to all flops.
<code>ena</code>	in	1	Tile enable. Gates <code>host_tx_valid</code> ; <code>ui_in</code> / <code>uio_in</code> are ignored when low.
<code>ui_in[7:0]</code>	in	8	Host→chip data byte (opcode in [7:4], field in [3:0] on header; payload on later bytes).
<code>uo_out[7:0]</code>	out	8	Chip→host data byte (response or LUT output).
<code>uio_in[2]</code>	in	1	<code>host_tx_valid</code> — <code>ui_in</code> is valid this cycle.
<code>uio_in[3]</code>	in	1	<code>host_rx_ready</code> — host has consumed <code>uo_out</code> .
<code>uio_in[1:0],[7:4]</code>	in	6	Reserved; driven low by host.
<code>uio_out[0]</code>	out	1	<code>chip_rx_ready</code> — chip will consume <code>ui_in</code> this cycle.

uio_out[1]	out	1	chip_tx_valid — uo_out is valid this cycle.
uio_out[2]	out	1	err_sticky — reserved-opcode or protocol error since last clear.
uio_out[3]	out	1	busy — FSM is not in S_IDLE.
uio_out[7:4]	out	4	state[3:0] — low nibble of FSM state for debug.
uio_oe[7:0]	out	8	Static 8'hF3. Not driven dynamically.

WHY PARTITION RATHER THAN REVERSE UIO

The Tiny Tapeout pad ring supports per-pin drive-enable, but a static partition removes a pile of glitch and contention risk without losing meaningful flexibility. The four wires the chip needs to drive (rx_ready, tx_valid, err_sticky, busy) and the four it needs to read (host's valid/ready) are known at design time, and a fixed mask is cheaper than per-cycle direction control.

Parameters

The synthesis-time parameters below are the union of the top-level wrapper and the leaf modules. Most are derived ($\$clog2$ of another parameter) and are listed here for completeness so a verification environment can mirror them without re-deriving.

Parameter	Default	Where	Description
ROWS	4	all	MAC array rows; also act_streamer fill depth.
COLS	4	all	MAC array columns; also bank write width.
DATA_W	4	all	Signed INT4 activation/weight width.
ACC_W	20	PE, bank, requant	PE accumulator and bank storage width. Sized for ~64 partial sums.
OUT_W	8	requant, lut	Post-requant output width (INT4 sign-extended into 8 bits, or INT8).
SHIFT_W	$\$clog2(ACC_W)+1 = 6$	requant, ctrl_io	Shift-amount width for requant; MODE[4:0] supplies it.
BANKS	2	bank	Ping-pong sub-tiles. Total bank depth = ROWS×COLS×BANKS = 32.
ROM_AW	8	rom, ctrl_io	256-entry weight ROM address width.
BNK_AW	$\$clog2(32) = 5$	bank, ctrl_io	Flat {bank, row, col} read address.
ARR_OUT	$ACC_W + \$clog2(ROWS) = 22$	mac2 array, bank	Array column-sum width. Bank saturates back to ACC_W on entry.
N_ENTRIES	16	act_lut	LUT depth.
FUNC	0 (EXP_NEG)	act_lut	Synth-time function select. 0/1/2/3 = EXP_NEG / IDENTITY / GELU.
INIT_SEED	32'hC0FFEE42	rom	LFSR seed for default ROM contents when no init file is supplied.

* GELU and SILU are reserved FUNC values that currently elaborate to the identity table. They are stubbed so the parameter interface is stable across future revisions.

ARR_OUT VS ACC_W

The MAC array sums four INT20 PE accumulators per column combinationally, producing a 22-bit column sum. The bank widens to 21 bits during the saturating accumulate add and then clamps back to INT20 storage. Saturation behavior is observable from software via the sticky overflow bit and is part of the contract.

Streaming Protocol & Opcode Set

All host↔chip traffic flows over a single ready/valid byte stream. Each transaction starts with a 1-byte header carrying a 4-bit opcode in *ui_in*[7:4] and a 4-bit field in *ui_in*[3:0]. The field carries either an immediate argument or a routing decision (which bank, which ROM page, whether the LUT is enabled); payload bytes, if any, follow on subsequent *host_tx_valid* cycles. Responses are emitted on *uo_out* gated by *chip_tx_valid* and consumed when the host raises *host_rx_ready*.

Handshake summary

Edge	Meaning
host_tx_valid & chip_rx_ready	Chip latches <i>ui_in</i> into the FSM this cycle.
chip_tx_valid & host_rx_ready	Host latches <i>uo_out</i> into its buffer this cycle.
ena = 0	<i>host_tx_valid</i> is masked off inside the chip; the FSM stays in S_IDLE.

Opcode set

Op	Name	Payload	Response	Effect
0x0	NOP	—	—	No state change.
0x1	RD_STATUS	—	1 B STATUS	Returns the STATUS CSR.
0x2	WR_CTRL	field	—	Writes CTRL. field[3] clears err and ovfl sticky bits.
0x3	WR_MODE	1 B	—	Writes the MODE CSR (shift, int8/int4, round, lut_default).
0x4	LOAD_W_ROM	—	—	field = 4-bit ROM page. Walks 16 ROM entries into the weight buffer.
0x5	LOAD_W_DIRECT	8 B (16 nibbles)	—	Loads the 4×4 weight buffer from the host, then pulses <i>mac_load_w</i> .
0x6	STREAM_ACT	2 B (4 acts)	—	field = {wr_mode, row[1:0], bank}. Streams one row through the array.
0x7	DRAIN_ACC	—	16 B	field = {2'h0, lut_en, bank}. Walks 16 entries through <i>requant_sat</i> (a).
0x8	RUN_TILE	—	—	field = {3'h0, bank}. Clears the named bank and the PE accumulator.
0x9	CLEAR_BANK	—	—	field = {3'h0, bank}. Clears the named bank only.
0xA	RD_ACC	1 B (low addr)	3 B sign-ext	field = high addr bits. Reads one INT20 entry from the bank, sign-ext.
0xB–0xF	reserved	—	—	Sets <i>err_sticky</i> in STATUS. No state change.

WHY OPCODES INSTEAD OF CSRS

A 4-bit opcode plus a 4-bit field fits naturally into the single 8-bit host→chip byte and avoids a separate address phase. Every byte sent to the chip is either an opcode header or an explicit payload byte demanded by the in-progress opcode. The four CSR-like registers (STATUS, CTRL, MODE, and the readable accumulator) live behind dedicated opcodes rather than a memory-mapped address space.

CSR Map

Three programmer-visible registers and one read-only status word.

STATUS — 8 bits, read via RD_STATUS

Bit	Name	Description
7	err_sticky	Set when a reserved opcode is decoded. Cleared by WR_CTRL with field[3]=1.
6	busy	Live mirror of (state $\neq$ S_IDLE). Not sticky.
5	ovfl_sticky	Set on any requant_sat clamp event. Cleared by WR_CTRL with field[3]=1.
4	reserved	Reads 0.
3:0	state[3:0]	Low nibble of the FSM state encoding (debug).

CTRL — 4 bits, written via WR_CTRL.field

Bit	Name	Description
3	clear_sticky	Pulses err_sticky and ovfl_sticky to 0 on the cycle WR_CTRL is decoded.
2:0	reserved	Ignored. Future allocation.

MODE — 8 bits, written via WR_MODE payload

Bit	Name	Description
7	lut_enable_default	Default LUT enable when DRAIN_ACC field[1] is 0.
6	requant_round	Round-half-up on requant when shift > 0.
5	requant_int8	0 = INT4 output (clamp $\pm[8, 7]$); 1 = INT8 (clamp $\pm[128, 127]$).
4:0	requant_shift	Arithmetic right-shift amount applied to the INT20 accumulator before clamping.

ATOMICITY

WR_CTRL takes effect on the same cycle its header byte is decoded; clear_sticky and the FSM transition are co-incident. WR_MODE takes effect one cycle later, when the payload byte is latched into mode_q. Software that wants a guaranteed deterministic MODE on the next STREAM_ACT should issue an intervening NOP or RD_STATUS, not back-to-back WR_MODE + STREAM_ACT.

Data-Plane Microarchitecture

MAC Array (`mac_array_4x4`)

16 instances of `int4_pe` arranged in a 4×4 grid. Each PE holds one signed INT4 weight in `weight_q`, multiplies the broadcast row activation by that weight, sign-extends the 8-bit product to `ACC_W`, and adds it into `acc_q`. PE control priority is `rst_n > load_weight > clear_acc > en`; in particular, `clear_acc` wins over the MAC update, so the controller's `RUN_TILE` clears both the named bank and the PE accs in a single cycle without racing the next `STREAM_ACT`.

On top of the per-PE registers, the array adds a column reduction stage: a combinational adder tree sums the four PE accumulators in each column to a 22-bit value, registered into `col_q`. This is the second pipeline stage and the array's output to the accumulator bank.

Accumulator Bank (`accum_bank`)

Storage is a **BANKS × ROWS × COLS = 2 × 4 × 4 = 32** entry array of signed INT20 cells. Each write cycle accepts a full column-vector (COLS entries) addressed by `{wr_bank, wr_row}`, so a full row of the MAC array's `col_flat_o` lands in one cycle.

<code>wr_mode</code>	Semantics
0 OVERWRITE	$\text{mem}[\text{bank}][\text{row}][c] \leftarrow \text{sat}(\text{in}[c])$. Used on the first pass of a tile.
1 ACCUMULATE	$\text{mem}[\text{bank}][\text{row}][c] \leftarrow \text{sat}(\text{mem}[\text{bank}][\text{row}][c] + \text{in}[c])$. Used on subsequent passes of the same output tile.

The accumulating add is widened to `ACC_W+1` bits, then saturated back to INT20 on the way into storage. Saturation events are not directly observable from the bank, but downstream `requant_sat` exposes its own sticky overflow.

Ping-pong usage. The two banks let software drain one 4×4 sub-tile out through `DRAIN_ACC` or `RD_ACC` while the other is still being accumulated by `STREAM_ACT`. The bank select bit comes from the opcode field on every relevant instruction (`STREAM_ACT`, `CLEAR_BANK`, `RUN_TILE`, `RD_ACC`), so the host steers which sub-tile is active.

Activation Streamer (`act_streamer`)

A 4-deep fill buffer with ready/valid push and a ROWS-wide pop. Bytes arriving from `ctrl_io` fill `buf_q[0..3]`; when `fill_cnt = 4` the buffer asserts `out_valid` and presents all four nibbles in parallel to the array's `act_flat_i`. A consumer pop empties the buffer and lets the next push land in slot 0 the same cycle, so back-to-back transactions do not stall. The controller also drives a `flush_i` input to reset `fill_cnt` between transactions without producing a spurious `out_valid`.

Requantizer (`requant_sat`)

Stateless shift-round-saturate. Computes $shifted = (data_i + bias) \ggg shift_i$, where $bias = round_i ? (1 \ll (shift_i - 1)) : 0$ implements round-half-up. The result is clamped to $[-8, +7]$ in INT4 mode or $[-128, +127]$ in INT8 mode. Any clamp event sets a sticky overflow bit that is surfaced in STATUS.ovfl_sticky; it clears only on reset or an explicit WR_CTRL with field[3]=1.

Activation LUT (act_lut)

16-entry table indexed by the top 4 bits of the INT8 input, converted from two's-complement to offset-binary by inverting the sign bit. FUNC picks the table at elaboration:

FUNC	Name	Table
0	EXP_NEG	round($127 \times \exp(-(k-8)/2)$) clipped to INT8. Lower half saturates at +127. Hard-coded; requires N_ENTRIES
1	IDENTITY	$(k-8) \ll 4$. Used for bypass testing and as the GELU/SILU fallback.
2	GELU	Reserved. Falls back to IDENTITY.
3	SILU	Reserved. Falls back to IDENTITY.

An enable_i input lets ctrl_io route the requant output around the table when running plain INT4/INT8 quantization. Bypass still goes through the output register, so the pipeline depth is constant whether the LUT is enabled or not.

Pipeline & Performance

The data-plane pipeline has five register stages between the host byte landing on *ui_in* and the LUT byte appearing on *uo_out*. Stage boundaries below are the names of the flops that capture each stage's output.

Stage	Captured in	What happens
1	act_streamer.buf_q	Activation byte latched into the row fill buffer.
2	int4_pe.acc_q	INT4×INT4 multiply and add into the PE INT20 accumulator.
3	mac_array.col_q	Column adder tree across 4 PEs; 22-bit column sum registered.
4	accum_bank.rd_data	Bank write (saturating to INT20) on STREAM_ACT, then a 1-cycle read latency on DRAIN_A
5a	requant_sat.data_o	Shift-round-saturate; sticky overflow updates.
5b	act_lut.data_o	LUT lookup or bypass; identical latency either way.

Steady-state STREAM_ACT throughput is bounded by the protocol, not by the array. STREAM_ACT consumes two payload bytes (4 activations) plus per-byte handshake overhead per row, then the FSM spends three states (S_SA_DRAIN1 / S_SA_DRAIN2 / S_SA_BANK_WR) flushing the array and writing the bank. The array itself is only active for one cycle per STREAM_ACT, so peak compute substantially exceeds sustained compute.

WHY THE FSM HAS DRAIN STATES

After the streamer presents the row, the PEs need one cycle to register the new acc_q, and the column tree needs another cycle to register col_q. Only then is col_flat_o the correct value to write into the bank. Skipping S_SA_DRAIN2 would write stale column sums; this is the single most subtle interaction in the control plane and is worth stating explicitly rather than discovering it during post-silicon bring-up.

First-order steady-state throughput

A STREAM_ACT row burst takes, optimistically, **1 (S_IDLE decode) + 2 × 2 (S_SA_RX_LO + S_SA_PUSH_HI per byte) + 3 (drain + bank write) = 8 cycles**, producing one bank row (4 INT20 partial sums). That is 16 INT4×INT4 MACs in 8 cycles, or 2 MAC/cycle sustained at 100% host-feed efficiency. At 25 MHz this is ~50 MMAC/s; at 50 MHz, ~100 MMAC/s.

First-order unloaded latency

Header byte landing on *ui_in* → first DRAIN_ACC byte on *uo_out*, for a single STREAM_ACT followed by a DRAIN_ACC of one entry:

Step	Cycles
------	--------

STREAM_ACT (header + 2 payload + drain + bank write)	~8
DRAIN_ACC header decode and bank read launch	1 (S_DA_RD)
Requant launch (S_DA_RQ)	1
Requant register + LUT register	2
TX byte ack	1
Total (header in → first byte out)	~13

The data-plane portion alone (stages 2–5b above) is 5 cycles; the rest is FSM and handshake overhead.